

Insert(element)

- Make space for new element, say newNode. $\rightarrow$ Node newNode = new Node();
- Store element in newNode's data. $\rightarrow$ newNode.data = element;
- Set newNode's next to empty. $\rightarrow$ newNode.next = null;
- if list is empty then $\rightarrow$ if (head == null){
 - Make newNode as first (and only) node of list. $\rightarrow$ head = newNode;
 - Stop $\rightarrow$ return;}

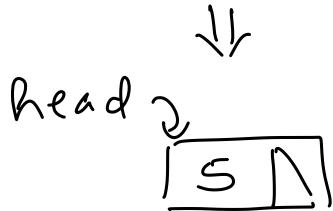
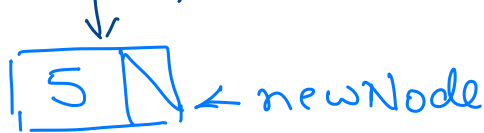
// List is not empty

// => Find first node having data greater than newNode's data.

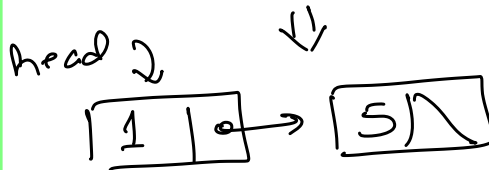
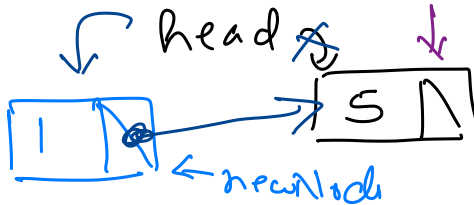
- Set current to first node. $\rightarrow$ current = head;
- Set previous to empty. $\rightarrow$ previous = null;
- while (current is not empty) do $\rightarrow$ while (current != null){
 - if (current node's data > newNode's data) then $\rightarrow$ if (current.data > newNode.data){
 - // Found the node
 - End the traversal. $\rightarrow$ break;
 - Set previous to current. $\rightarrow$ previous = current;
 - Move current to current's next node. $\rightarrow$ current = current.next;}

- ```
// Add newNode between previous and current
- Set newNode as next of previous. → previous.next = newNode;
- Set current as next of newNode. → newNode.next = current;
- Stop.
```

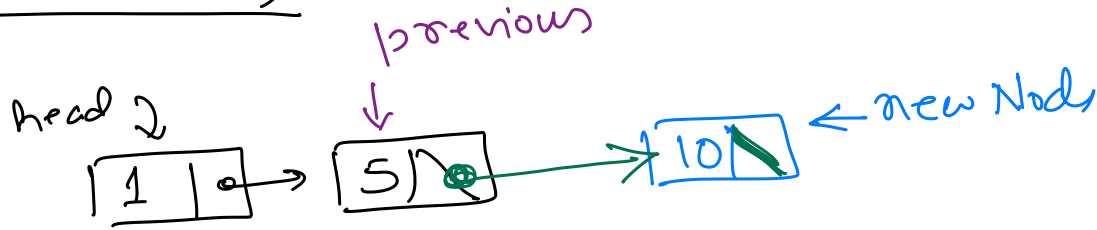
head ~~→~~ empty



insert (1)      previous  $\rightarrow$  empty  
current

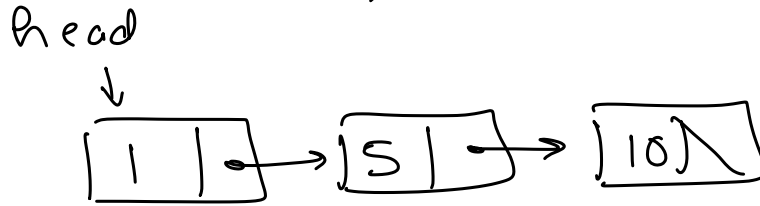


insert (10)



current  $\rightarrow$  empty

⇓



Exercise: Implement insert for a sorted singly linked list that uses head and tail pointers.

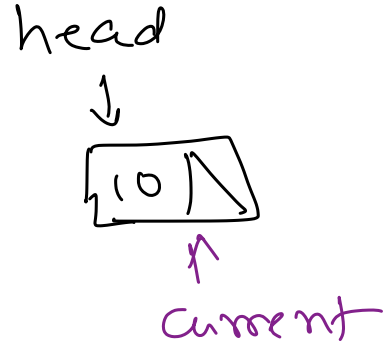
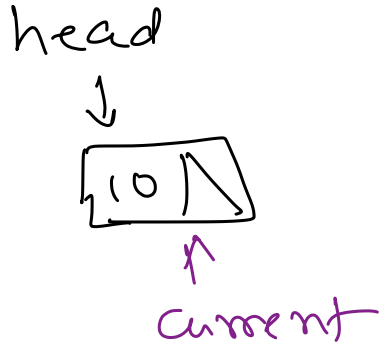
Insert in a list using Single pointer (current).

① Do not let current become empty.

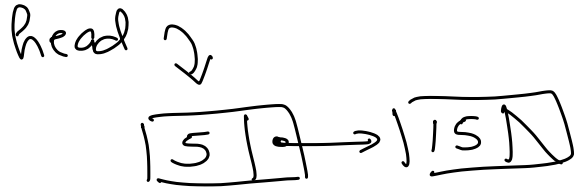
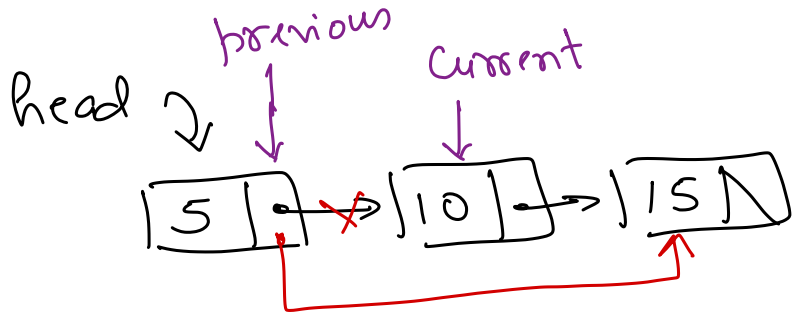
⇓

Stop traversal at last node.

while (current.next != null)



## Delete a node



## delete (10)

Set previous node's  
next to current  
node's next

## Special case

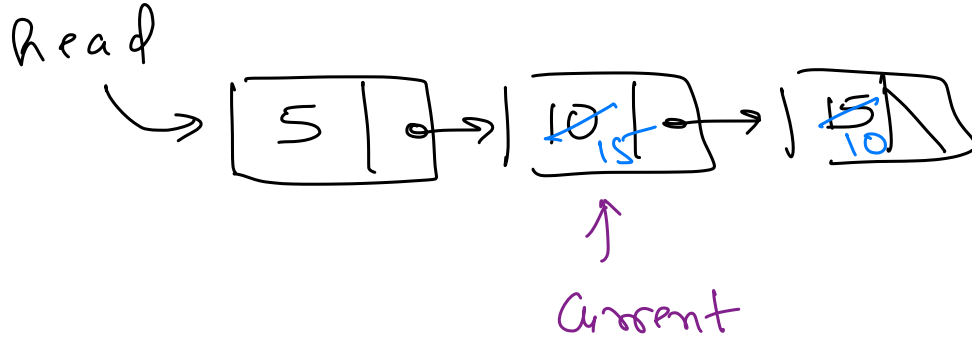
- ① Empty list
- ② list has only one node.
- ③ Element not found?
- ④ Delete first node?

Delete( element )

- Set current to first node (head)
- Set previous to empty.
- while (current is not empty) do
  - if (current node's data = element) then
    - End the traversal. // Element found
  - Set previous to current node.
  - Move current to current node's next node.
- if (current is empty) then // Element not present in list
  - Stop
- if (current node is first node of list) then // Deleting first node
  - Move head to head's next node.
  - Mark current node as free.
  - Stop.
- Set previous node's next to current node's next.
- Mark current node as free.
- Stop.

## Delete a given node

deleteNode(  
Node current).



① Swap current node and its next nodes values.

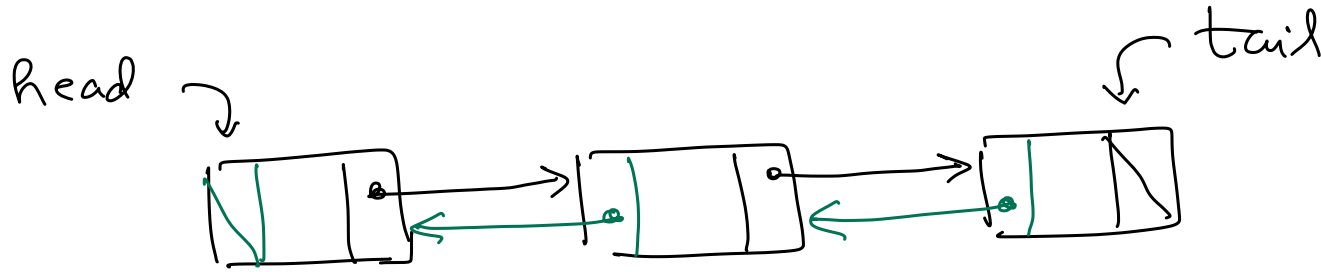
② Remove current's next node.

Can be taken  
care if list has  $\Leftarrow$   
a dummy node after last data node.

will not work if  
current is last node.

## Doubly Linked List

linked list in which each node keep track of its two adjacent nodes.



```
class Node {
 int data;
 Node next;
 Node previous;
}
```

Traversal → Forward traversal ← Same as singly list traversal.  
↳ Backward / Reverse traversal

↓  
Starts with last node and move backwards.

### Forward Traversal

- Set current to first node of list.
- while (current is not empty) do
  - Process current node.
  - Set current to current node's next.
- Stop.

### Backward Traversal

- Set current to last node of list.
- while (current is not empty) do
  - Process current node.
  - Set current to current node's previous .
- Stop.

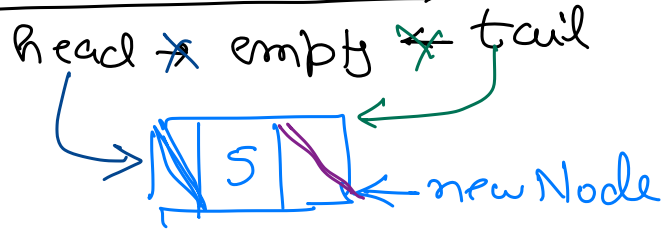
AddAtFront( element )

- Make space for new element, say newNode.
- Store element in newNode's data.
- Set previous of newNode as empty.
- if list is empty then
  - Make newNode as last node.

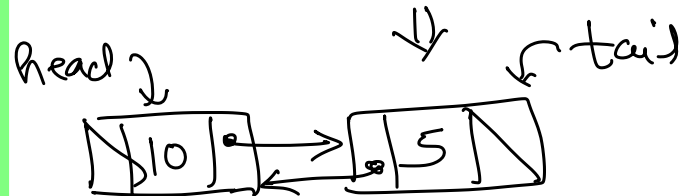
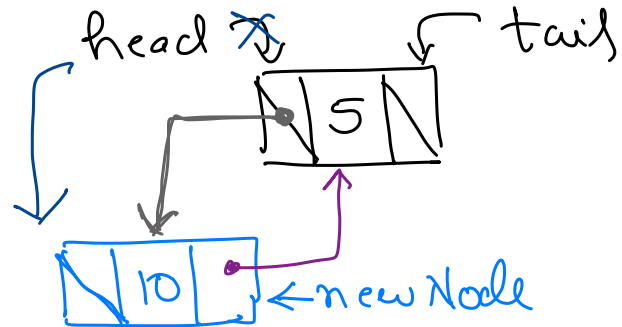
Else

- Set newNode as previous of head.
- Set newNode's next to head.
- Set head to newNode.
- Stop.

addAtFront (5)

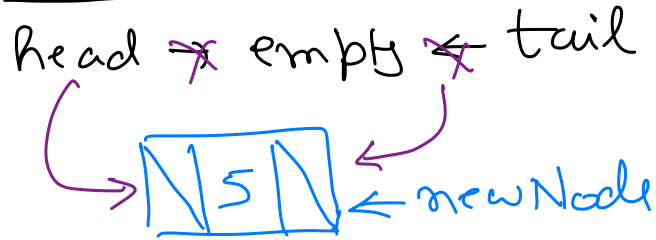


add At Front (10)

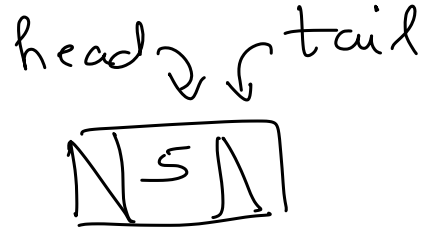


## add At End

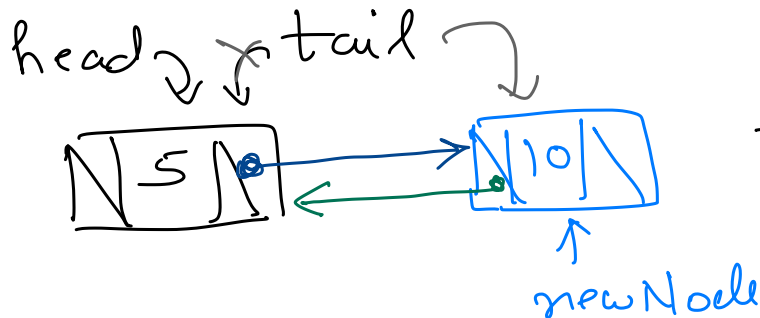
add At End (5)



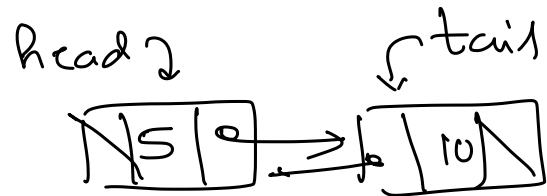
⇒



add At End (10)



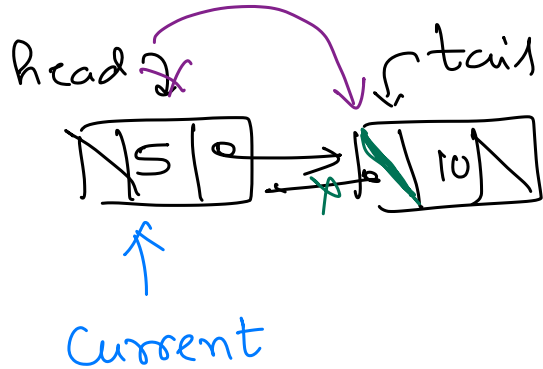
⇒



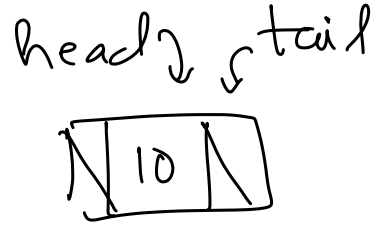
AddAtEnd( element )

- Make space for new element, say newNode.
  - Store element in newNode's data.
  - Set newNode's next to empty.
  - Set previous of newNode as empty.
- if list is empty then
- Set head to newNode.
  - Set tail to newNode.
  - Stop.
- Set newNode as next of tail node.
- Set previous of newNode to tail.
  - Set tail to newNode.
  - Stop.

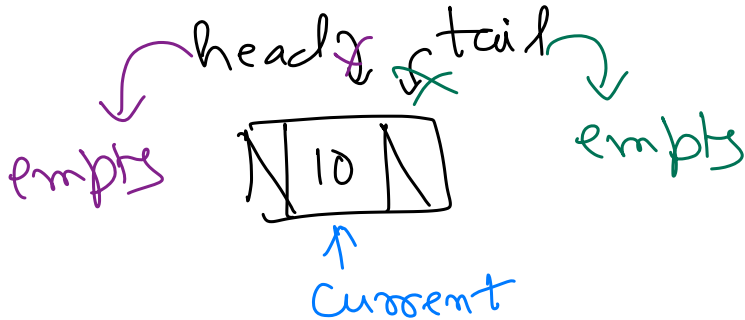
## delete First Node()



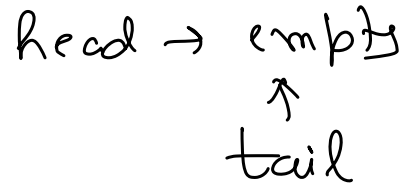
$\Rightarrow$



## add First Node()



$\Rightarrow$



DeleteFirstNode()

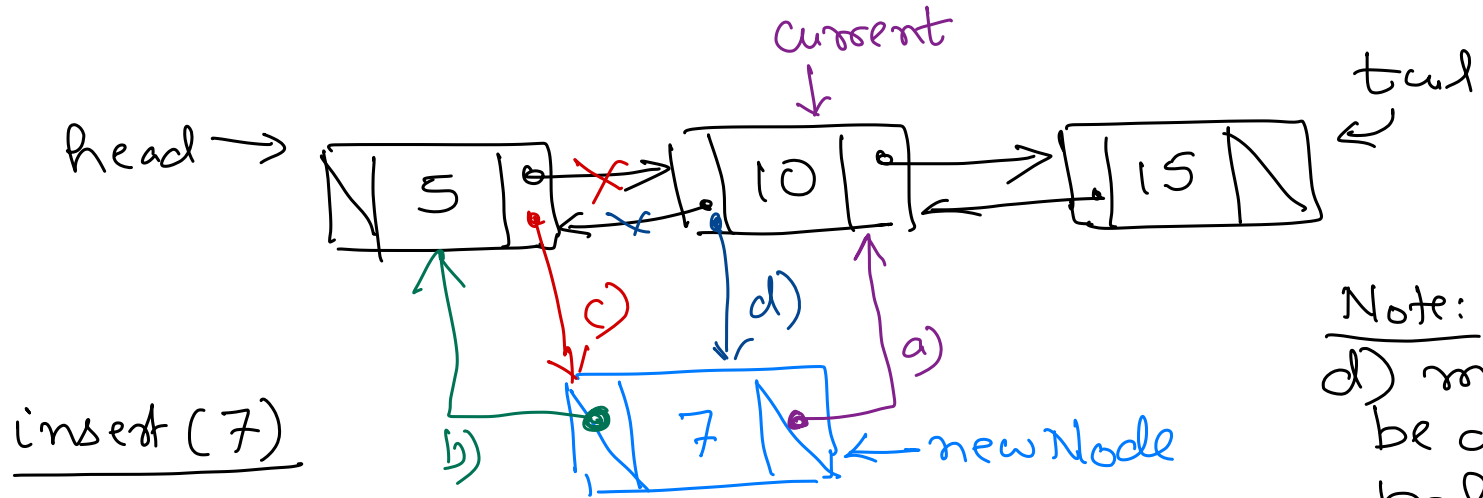
- if list is empty then
  - Stop.
- Set current to head
- Move head to head's next node.
- if list is empty then // Check if head is empty, as head got changed
  - Set tail to empty

Else // list is not empty

- Set previous of head to empty
- ~~Release current node.~~ ← Not needed in JAVA.
- Stop.

Add element to doubly list  $\Rightarrow$  insert.

$\rightarrow$  Create a sorted doubly list.



Note:

d) must not be done before b) and c).

a) Set new Node's next to current  
 $\text{newNode} \cdot \text{next} = \text{current};$

b) Set new Node's previous to current's previous.

$\text{newNode}.\text{previous} = \text{current}.\text{previous}$

c) Set current node's previous node's next to new Node.

$\text{current}.\text{previous}.\text{next} = \text{newNode};$

d) Set current's previous to newNode.

$\text{current}.\text{previous} = \text{newNode};$

Special cases

① List is empty.  $\Rightarrow$  set newNode as first and last node.

② Adding smallest value.  $\Rightarrow$  add newNode as first node.

③ Adding largest value  $\Rightarrow$  add newNode as last node.

Insert( element )

// 1. Create new node

- Make memory for new element, say newNode.
- Store element in newNode's data.
- Set newNode's next and previous to empty.

// 2. If list is empty?

- if head is empty then

// Make newNode as the first and last node of the list.

- Set head and tail to newNode.
- Stop.

// 3. Traverse list to find node - current node.

- Set current to head (first node).
- while (current is not empty) do
  - if (current node's data > element) then
    - // Found the node, end the traversal.
  - End the traversal.
- Set current to current's next node.

// 4. If adding before the first node? - Current is the first node.

- if (current is head) then

- Before the first node comes newNode. // Set head's previous to newNode.
- After newNode comes the first node. // Set newNode's next to head.
- Make newNode as the first node. // Set head to newNode.
- Stop.

// 5. If adding after the last node? - Current is empty

- if (current is empty) then

- After the last node comes newNode. // Set tail's next to newNode.
- Before newNode comes the last node. // Set newNode's previous to tail.
- Make newNode as the last node. // Set tail to newNode.
- Stop.

// 6. Add a new node between current and current's previous node.

- Make the current node come after newNode. // **Set newNode's next to current.**
- Make the current node's previous node come before newNode. // **Set newNode's previous to current node's previous.**
- Make newNode come after the current node's previous node. // **Set current node's previous node's next to newNode.**
- Make newNode come before the current node. // **Set current node's previous to newNode.**
- Stop.